Digital Safety Audit Report — Logfire Python SDK

Date: 2025-07-14
Auditor: Digital Safety Agent (SDL-aligned)
Project: `logfire` v4.31.0 — Open-source Python observability SDK
Scope: SDK source code, CLI, auto-instrumentation, scrubbing subsystem, LLM integrations, exporter pipeline
Out of Scope: Closed-source Logfire SaaS platform, backend storage, web dashboard
Methodology: Microsoft SDL Design- and Requirements-phase digital safety review
Context Documents: `audit/ARCHITECTURE.md`, `audit/THREAT_MODEL.md`

Executive Summary

The Logfire SDK is a developer-facing observability library with no end-user UI. Classical digital safety concerns (addictive design, photosensitivity, age-gating) are **not applicable**. The relevant safety domain here is **SDK-level digital safety**: the risks that arise from the SDK itself collecting, exporting, and retaining data from the host application and its users — potentially without clear developer awareness or user consent.

Seven findings were identified. The two highest-severity findings centre on the **deliberate exemption of LLM prompt/response content and SQL statements from the scrubbing pipeline**, which means that sensitive user-generated content and database query text is transmitted to the Logfire backend verbatim and without redaction. A third significant finding is the **silent disablement of all scrubbing** via `scrubbing=False` with no warning. Together, these create a pattern where the default or easily-configured SDK behaviour can silently transmit user PII and business-sensitive data to a third-party SaaS backend without explicit developer awareness of the risk.

ID	Severity	Title
DS-01	**High**	LLM input/output attributes are exempt from scrubbing by design
DS-02	**High**	SQL statement attribute (`db.statement`) is exempt from scrubbing
DS-03	**High**	`scrubbing=False` silently disables all PII redaction with no warning
DS-04	**Medium**	SDK sends host application metadata without prominent disclosure
DS-05	**Medium**	Pydantic plugin activates implicitly on `import pydantic`
DS-06	**Medium**	No rate-limiting or volume controls on span generation
DS-07	**Medium**	DiskRetryer accumulates up to 512 MB of telemetry with no shutdown cleanup
DS-08	**Low**	No pre-send data inventory or first-run disclosure mechanism
DS-09	**Low**	Console exporter prints span attributes including HTTP URLs in shared environments

---

Compulsive Design Inventory

Pattern	Present?	Notes
Reward loops / streaks	❌ No	SDK has no user-facing gamification
Variable-ratio rewards / loot boxes	❌ No	N/A
Loss-aversion mechanics	❌ No	N/A
Infinite scroll / endless content	❌ No	N/A

Pattern	Present?	Notes
Auto-play media	❌ No	N/A
Artificial urgency / countdown timers	❌ No	N/A
Re-engagement push notifications	❌ No	N/A
Session length limits / fatigue warnings	❌ No	N/A (developer tool)
Default behaviors encouraging over-instrumentation	⚠ Partial	`install_auto_tracing()` has no built-in volume guardrails (see DS-06)

No compulsive design patterns detected. SDK is a developer library without an end-user interaction model.

Dark Pattern Checklist

Pattern	Present?	Notes
Confirmshaming opt-out labels	❌ No	N/A
Roach motel (easy subscribe, hard cancel)	⚠ Partial	`send_to_logfire=False` exists but the path to disable is less prominent than the onboarding path; no "what you agreed to" summary shown
Hidden costs	❌ No	N/A (SDK free; SaaS pricing is external)
Misdirection / "recommended" option manipulation	❌ No	N/A

Pattern	Present?	Notes
Privacy zuckering / complex consent flows	⚠ Partial	No consent banner or data inventory shown before first telemetry transmission (see DS-08)
Disguised ads / sponsored content	❌ No	N/A
Forced continuity (auto-renewing trials)	❌ No	N/A
Implicit telemetry opt-in	⚠ Present	Pydantic plugin auto-activates on package installation (see DS-05); LLM content captured without `capture_input=True` flag requirement (see DS-01)

Photosensitivity Assessment

Result: PASS — No concerns identified.

- Console output uses the `rich` library for static styled terminal text (colours, tables, progress indicators).
- No animations, flashing, strobe patterns, or rapid alternating visuals in any code path.
- No auto-playing media, GIFs, or video content.
- WCAG 2.3.1 (< 3 flashes/second) is not applicable to a terminal SDK.
- No web UI is present in this repository; the closed-source dashboard is out of scope.

Age-Appropriateness Assessment

Result: N/A — Developer SDK.

- The SDK is exclusively targeted at Python developers building backend applications.
- No user-generated content, social features, direct messaging, or consumer-facing UI are present.
- No age verification is required or applicable.
- COPPA, GDPR Art. 8, and UK Children's Code are not applicable to this component.

Social & Communication Safety Assessment

Result: N/A — No social features.

- No public profiles, follower counts, likes, shares, or comment systems.
- No direct messaging or user-to-user communication.
- No user-generated content moderation surface.

Detailed Findings

[HIGH] DS-01 — LLM Input/Output Attributes Exempt from Scrubbing by Design

- **SDL Phase**: Design
- **Category**: SDK Telemetry / Implicit Consent

- **File**: `logfire/_internal/scrubbing.py` (lines 148–160), `logfire/_internal/integrations/llm_providers/openai.py` (lines 132–151), `logfire/_internal/integrations/llm_providers/semconv.py`

- **Description**:
  The `BaseScrubber.SAFE_KEYS` set explicitly excludes the following attributes from all scrubbing:

  ```
  'gen_ai.input.messages',
  'gen_ai.output.messages',
  'gen_ai.system_instructions',
  'pydantic_ai.all_messages',
  ```

  These attributes contain the **full verbatim text** of LLM prompts, system messages, and model responses captured by `instrument_openai()`, `instrument_anthropic()`, `instrument_pydantic_ai()`, and all other AI framework instrumentation. By marking these as safe, the scrubbing pipeline is intentionally bypassed for all LLM content.

  Concretely, when a developer calls `logfire.instrument_openai(client)`, every chat completion request — including the messages array containing the user's input text — is captured in the `gen_ai.input.messages` span attribute and exported to the Logfire SaaS backend verbatim, with no pattern-based redaction applied. If a user asks an AI assistant "What are my account number and password?", that string is exported.

  The threat model (TM-04) identifies this as **Critical** severity. The scrubbing code was clearly written to prevent double-scrubbing of already-structured AI content, but the practical effect is that any PII, credentials, business-sensitive information, or user conversation data embedded in an LLM interaction bypasses all redaction.

  Additionally, the OpenAI instrumentation at `get_endpoint_config()` includes `request_data=json_data` (line 133) — the **full raw JSON request body** — as a span attribute by default when using semconv version 1. This captures all request parameters alongside the structured message content.

- **Affected Users**: End users of any application instrumented with Logfire's AI integrations. Their conversation text, queries, and any PII embedded in LLM interactions is silently exported to a third-party SaaS backend. Developers who have not audited the

`SAFE_KEYS` list may not be aware of this behaviour.

- **Regulatory Reference**: GDPR Art. 5(1)(c) (data minimisation), Art. 13/14 (transparency), UK Online Safety Act 2023 (safety by design), EU AI Act (transparency requirements for AI systems). For applications serving users in regulated sectors (healthcare, finance), this also touches HIPAA, PCI-DSS.
- **Recommendation**:
  i. **Opt-in capture**: Gate LLM content capture behind explicit `capture_input=True` / `capture_output=True` flags that default to `False`. This is the approach recommended in Mitigation M-02 of the threat model and mirrors industry practice (e.g., OpenTelemetry Semantic Conventions for GenAI mark these attributes as opt-in).
  ii. **Remove from SAFE_KEYS**: The rationale for bypassing scrubbing is to prevent re-scrubbing of already-structured data. A better approach is to apply scrubbing to the *string values within* these structured attributes (e.g., scan message content strings against the scrubbing pattern) rather than exempting them entirely.
  iii. **Document prominently**: If opt-in capture is added, the documentation for `instrument_openai()` etc. should contain a prominent notice that enabling content capture logs user conversation data to the Logfire backend.
  iv. **Emit a `UserWarning`** when LLM content capture is enabled, at the call site.

[HIGH] DS-02 — SQL Statement Attribute Exempt from Scrubbing

- **SDL Phase**: Design
- **Category**: SDK Telemetry / Implicit Consent
- **File**: `logfire/_internal/scrubbing.py` (line 137)
- **Description**:
  The attribute `db.statement` is listed in `BaseScrubber.SAFE_KEYS`:

```
'db.statement',
'db.plan',
```

This means all SQL (and other query language) statements captured by DB instrumentation — `instrument_sqlalchemy()`, `instrument_asyncpg()`, `instrument_psycopg()`, `instrument_sqlite3()`, `instrument_mysql()`, `instrument_pymongo()` — are exported to the Logfire backend **without any scrubbing applied to the statement text**.

Many ORM-level or dynamically constructed queries include inline parameter values that may contain PII (e.g., `SELECT * FROM users WHERE email = 'alice@example.com'`, string-interpolated WHERE clauses, stored procedure calls with user-supplied arguments). While parameterised queries keep values separate from the statement text, this depends entirely on how the application constructs queries. The SDK cannot know whether a given `db.statement` string contains sensitive values; by blanket-exempting it from scrubbing, the SDK assumes it is always safe.

The threat model (TM-05) rates this as **High**. The exemption was presumably added to prevent false positives (e.g., scrubbing SQL keywords like `SESSION` or `AUTH` that match scrubbing patterns) but the result is that the attribute is wholly unprotected.

- **Affected Users**: Developers whose applications pass user-supplied data through string-formatted SQL. End users whose PII is embedded in captured query text.
- **Regulatory Reference**: GDPR Art. 5(1)(c) (data minimisation), HIPAA for healthcare applications, PCI-DSS for payment-related queries.
- **Recommendation**:
  i. **Remove `db.statement` from SAFE_KEYS** and instead apply scrubbing to the statement text with targeted SQL-aware patterns that avoid false-positive matches on SQL keywords.
  ii. Alternatively, add a `capture_db_statement=True/False` flag to DB instrumentation methods (defaulting to `True` for backward compatibility but `False` in a future major version), allowing operators to explicitly disable statement capture.
  iii. At minimum, add a documentation notice that `db.statement` is transmitted verbatim and may contain user-supplied data.

[HIGH] DS-03 — `scrubbing=False` Silently Disables All PII Redaction

- **SDL Phase**: Design
- **Category**: Dark Pattern / SDK Safety
- **File**: `logfire/_internal/config.py` (lines 736–741), `logfire/_internal/scrubbing.py` (lines 172–183)
- **Description**:
  When a developer calls `logfire.configure(scrubbing=False)`, the SDK replaces the default `Scrubber` with a `NoopScrubber` (a no-operation implementation). From that point forward, **all span attributes, log record bodies, event attributes, and link attributes are exported verbatim** to the Logfire backend, bypassing every default pattern match.

  The code path is:

  ```
  # config.py lines 739-741
  self.scrubbing: ScrubbingOptions | Literal[False] = scrubbing
  self.scrubber: BaseScrubber = (
      Scrubber(scrubbing.extra_patterns, scrubbing.callback) if scrubbing else NOOP_SCRUBBER
  )
  ```

  There is no `warnings.warn()`, no log message, no confirmation prompt, and no documentation notice printed at runtime. A developer who adds `scrubbing=False` as a "quick debug fix" to diagnose a scrubbing false-positive and then forgets to remove it will silently export all credentials, session tokens, and PII in their span attributes to the backend — indefinitely.

  This also applies to the `LOGFIRE_SCRUBBING=false` environment variable equivalent (if such exists via `config_params.py` — the pattern for other config params suggests this is configurable).
- **Affected Users**: Developers and operators who disable scrubbing (intentionally or accidentally). End users of those applications whose data is then exported.
- **Regulatory Reference**: GDPR Art. 25 (data protection by default), UK GDPR, CCPA.

- **Recommendation**:
  i. **Emit a `SecurityWarning`** (using `warnings.warn(..., SecurityWarning, stacklevel=2)`) at the call site when `scrubbing=False` is passed to `configure()`. Include a message explaining that all PII scrubbing is disabled and that span attributes will be exported verbatim.
  ii. **Repeat the warning once per process** in the first span export with scrubbing disabled (using a flag similar to `_should_log()`), so that the warning appears in logs even if `warnings` are filtered.
  iii. Consider adding a `logfire.configure(scrubbing=ScrubbingOptions(unsafe_disable=True))` pattern that makes the intent explicit and auditable in code review.
  iv. Document `scrubbing=False` with a prominent security caution block in the API reference and quickstart.

[MEDIUM] DS-04 — SDK Sends Host Application Metadata Without Prominent Disclosure

- **SDL Phase**: Requirements
- **Category**: SDK Telemetry / Implicit Consent
- **File**: `logfire/_internal/config.py` (lines 947–982), `logfire/_internal/collect_system_info.py`
- **Description**:
  On every call to `logfire.configure()` (and subsequently with every exported telemetry payload), the SDK automatically collects and transmits the following as OTel resource attributes in the OTLP export:

Attribute	Value	Notes
`service.name`	Application name	Set by developer or env var
`process.pid`	OS process ID	Automatically collected

Attribute	Value	Notes
`process.runtime.name`	e.g., `cpython`	Automatically collected
`process.runtime.version`	Python version string	Automatically collected
`process.runtime.description`	Full Python `sys.version`	Automatically collected
`service.version`	Git revision hash (if available)	Automatically derived from git
`service.instance.id`	Random UUID per process	Generated on init
`code.work_dir`	Current working directory	Only when `code_source` is set
`deployment.environment.name`	Environment label	If configured

A commented-out line (`# RESOURCE_ATTRIBUTES_PACKAGE_VERSIONS: json.dumps(collect_package_info(), ...)`) shows that full package inventory collection was considered and is implemented via `collect_package_info()`, which enumerates all installed packages via `importlib.metadata`. While currently disabled, this function remains callable and is not removed.

These attributes are transmitted with **every span batch**, not just once. A developer integrating Logfire into a production application may not realise they are continuously broadcasting their Python version, process PID, service instance UUID, and git commit hash to a third-party SaaS backend as a side-effect of adding observability.

This is standard OTel practice, but the **absence of a disclosure mechanism** (a "what will be collected" summary at first configure, or documentation callout on the getting-started page) means developers may not have consciously consented to metadata collection.

- **Affected Users**: Developers and companies whose infrastructure metadata is exported. In regulated environments (government, finance, healthcare), process-level metadata can constitute sensitive configuration information.
- **Regulatory Reference**: GDPR Art. 13 (information to be provided), principle of transparency.

- **Recommendation**:
  i. Add a **first-run disclosure** in the console output (the project link summary already prints on first configure) listing the resource attributes that will be collected and sent.
  ii. Publish a clear **data inventory** in the documentation: exactly what is collected by default, what requires opt-in, and how to opt out of each category.
  iii. Consider making `process.pid` and `process.runtime.description` opt-out rather than always-on, or providing a `resource_attributes_filter` parameter.
  iv. The commented-out `collect_package_info()` call should either be removed or documented as a non-default opt-in feature, to prevent future accidental re-enabling.

[MEDIUM] DS-05 — Pydantic Plugin Activates Implicitly on `import pydantic`

- **SDL Phase**: Design
- **Category**: Implicit Consent / SDK Safety
- **File**: `logfire/integrations/pydantic.py` (lines 27–28, 302–361), `logfire/_internal/config_params.py` (line 94), `pyproject.toml` (`[project.entry-points."pydantic"]`)
- **Description**:
  The Logfire Pydantic plugin is registered as a `[project.entry-points."pydantic"]` entry point. Pydantic v2's plugin system activates all registered plugins automatically whenever `pydantic` is imported — which occurs before `logfire.configure()` is called in most applications.

  At module import time, the plugin file executes this code unconditionally:

```
# pydantic.py lines 27-28
METER = GLOBAL_CONFIG._meter_provider.get_meter('logfire.pydantic')
validation_counter = METER.create_counter('pydantic.validations')
```

This attaches to the global `GLOBAL_CONFIG` before the developer has called `logfire.configure()`. The `new_schema_validator` method is subsequently called for every Pydantic model defined in the application (at class creation time), even if the developer never intended to use Logfire.

While the default `pydantic_plugin_record='off'` means validation *data* is not recorded until explicitly enabled, the counter metric instrumentation and the wrapping of all validator functions still occurs. More critically, any developer who has `logfire` installed as an indirect dependency (e.g., a library that depends on `logfire-api`) will have this plugin fire silently.

This violates the **principle of explicit opt-in** for instrumentation. The threat model (TM-14) identifies this as Medium severity. Unlike most instrumentation calls, there is no `logfire.instrument_pydantic()` gating — the effect is automatic on installation.

- **Affected Users**: Any Python application that has `logfire` installed (directly or transitively) and uses Pydantic — even if the application developer has not configured Logfire. The plugin modifies the validator wrapping for all Pydantic models globally.
- **Regulatory Reference**: GDPR Art. 25 (data protection by design and by default), principle of minimal footprint.
- **Recommendation**:
  i. **Add an explicit opt-in gate**: The Pydantic plugin's `new_schema_validator` should immediately return `(None, None, None)` unless `logfire.configure()` has been called (i.e., check `GLOBAL_CONFIG._initialized`). This already happens for recording, but the counter meter is still created at import time.
  ii. Move `METER` and `validation_counter` creation inside the plugin activation path (after `logfire.configure()` is verified) rather than at module import time.
  iii. Consider adding a check in the plugin entry point: if `LOGFIRE_PYDANTIC_PLUGIN_RECORD` is not set (not just defaulting to `'off'`, but *not configured at all*), the plugin should self-deactivate entirely, requiring explicit `LOGFIRE_PYDANTIC_PLUGIN_RECORD=all` to activate.

iv. Document the auto-activation behaviour prominently in the Pydantic integration page with an explicit note that installing `logfire` changes the global Pydantic validator wrapping.

[MEDIUM] DS-06 — No Rate Limiting or Volume Controls on Span Generation

- **SDL Phase**: Requirements
- **Category**: SDK Safety / Resource Exhaustion
- **File**: `logfire/_internal/auto_trace/__init__.py`, `logfire/_internal/exporters/dynamic_batch.py`, `logfire/_internal/exporters/otlp.py`
- **Description**:
  The SDK provides no built-in mechanism to limit the rate or volume of span generation. Specific risk scenarios:

  i. **`install_auto_tracing()` with a broad module filter** (e.g., matching all non-stdlib modules) instruments every function entry/exit. In a busy web application, this can generate hundreds of thousands of spans per second, saturating the in-memory `BatchSpanProcessor` queue, the `DiskRetryer` (up to 512 MB), and network bandwidth.

  ii. **No circuit breaker**: If the Logfire backend returns repeated 429/503 responses, the `DiskRetryer` accumulates payloads indefinitely up to the 512 MB cap. There is no automatic back-off that reduces the span generation rate at the source — the SDK continues generating spans at the same rate, consuming memory and CPU.

  iii. **No per-key cardinality limits**: High-cardinality span attributes (e.g., user IDs, request IDs in attribute names) are not detected or warned about.

  The `BodySizeCheckingOTLPSpanExporter` enforces a 5 MB per-request limit (splitting large batches), and `DynamicBatchSpanProcessor` provides dynamic batching, but these are throughput optimisations, not safety guardrails. The `sampling.head` parameter provides head-based sampling but requires the developer to configure it proactively.

- **Affected Users**: Developers who use `install_auto_tracing()` with broad filters on high-throughput applications. Operations teams who may face unexpected disk exhaustion or network saturation.
- **Regulatory Reference**: NIST SP 800-53 SI-17 (fail-safe procedures), general software engineering principle of graceful degradation.
- **Recommendation**:
  i. Add a **configurable spans-per-second warning threshold** to `install_auto_tracing()` (e.g., `max_spans_per_second=10_000`): emit a `UserWarning` when the observed rate exceeds the threshold, suggesting the user tighten their module filter or increase `min_duration`.
  ii. Add a **circuit breaker mode** to `DiskRetryer`: when the disk queue exceeds a configurable fraction of `MAX_TASK_SIZE`, reduce (or suspend) the in-memory queue acceptance rate to apply back-pressure at the source.
  iii. Expose a `logfire.configure(sampling=SamplingOptions(head=0.1))` recommendation in the `install_auto_tracing()` documentation and docstring.
  iv. Consider adding a `max_queue_size` parameter to `DynamicBatchSpanProcessor` that drops spans (with a counter metric and logged warning) when the queue is full, rather than blocking.

[MEDIUM] DS-07 — DiskRetryer Accumulates Up to 512 MB of Telemetry with No Shutdown Cleanup

- **SDL Phase**: Design
- **Category**: SDK Safety / Data Retention
- **File**: `logfire/_internal/exporters/otlp.py` (lines 105–221)
- **Description**:
  When the Logfire backend is unreachable, the `DiskRetryer` persists failed OTLP export payloads to disk under a

`mkdtemp(prefix='logfire-retryer-')` directory. These files are raw Protobuf bytes (post-scrubbing) of span/log/metric data. Up to 512 MB may accumulate (`MAX_TASK_SIZE`).

Two digital safety concerns arise:

i. **No cleanup on process shutdown**: The retry daemon thread is a `daemon=True` thread. When the main process exits, the thread is killed and all pending retry files are left on disk indefinitely in `/tmp/logfire-retryer-*/`. The directory is never deleted on normal shutdown. If the backend remains unreachable for extended periods, multiple process restarts will accumulate multiple `logfire-retryer-*` directories.

ii. **Telemetry data persistence**: Span payloads may contain HTTP URLs, SQL statements, exception messages, function arguments (from auto-trace), and LLM content. While scrubbing runs before export, the gaps identified in DS-01 and DS-02 mean this data may include user PII that persists on disk indefinitely.

A developer testing locally who sets `scrubbing=False` for a debugging session will have all span data (including credentials and tokens in span attributes) persisted in `/tmp` until manually cleaned.

- **Affected Users**: Any user of the application whose data appears in span attributes. System administrators managing disk space on application hosts.
- **Regulatory Reference**: GDPR Art. 5(1)(e) (storage limitation), UK GDPR, CCPA.
- **Recommendation**:

  i. **Register a shutdown hook** (`atexit.register`) in `DiskRetryer.__init__` that attempts a final retry flush with a short timeout, then **deletes all remaining retry files** and the directory on process exit.

  ii. Add a **per-directory age limit**: on startup, `DiskRetryer` should scan for and delete any `logfire-retryer-*` directories older than a configurable maximum age (e.g., 24 hours), to prevent orphaned data from accumulating across restarts.

  iii. Tighten the directory permissions: ensure `mkdtemp` creates a `0700` directory on all platforms (verify Windows behaviour).

[LOW] DS-08 — No Pre-Send Data Inventory or First-Run Disclosure Mechanism

- **SDL Phase**: Requirements
- **Category**: Transparency / Dark Pattern
- **File**: `logfire/_internal/config.py` (lines 1061–1122), `logfire/_internal/auth.py`
- **Description**:
  When a developer calls `logfire.configure(token=...)` or runs `logfire auth` for the first time, the SDK immediately begins sending telemetry to the Logfire SaaS backend. There is no mechanism that:

  - Lists what categories of data will be collected and exported.
  - Explains which integrations (if active) capture what content (HTTP bodies, SQL statements, LLM prompts).
  - Provides a "dry-run" or preview mode that shows what would be exported without actually sending.
  - Logs an entry indicating that data export has started.

  The console output on configure prints a project URL (the "project link summary"), which is a positive signal that data is flowing. However, it does not specify *what* data is flowing or alert the developer to any high-sensitivity captures (e.g., "LLM content capture is active, user prompts will be sent to Logfire").

  For a SaaS-connected observability SDK used in production applications, this creates a transparency gap: the developer may have instrumentated an OpenAI client, enabled auto-tracing, and disabled scrubbing without realising the combined effect is that all user conversation data and function arguments containing credentials are being exported.
- **Affected Users**: Developers integrating Logfire into production applications. Downstream users of those applications.
- **Regulatory Reference**: GDPR Art. 13 (information to be provided at collection), Art. 25 (data protection by design).
- **Recommendation**:

i. At the end of `LogfireConfig._initialize()`, **print a data collection summary** to the console (respecting `LOGFIRE_CONSOLE=false`): list active integrations, whether LLM content capture is enabled, scrubbing status, and a pointer to documentation explaining what each integration captures.
ii. Add a `logfire.what_will_be_collected()` utility function that returns a structured summary of the current configuration's data collection scope.
iii. When `scrubbing=False` is combined with any AI instrumentation, print a prominent warning at configure time (beyond the warning recommended in DS-03).
iv. Add a note to the getting-started documentation: "Before going to production, review [data collection guide] to understand what your application sends to Logfire."

[LOW] DS-09 — Console Exporter Prints Span Attributes Including HTTP URLs in Shared Environments

- **SDL Phase**: Design
- **Category**: Information Disclosure
- **File**: `logfire/_internal/exporters/console.py`, `logfire/_internal/config.py` (lines 1038–1055)
- **Description**:
  The console exporter is **enabled by default** (`CONSOLE = ConfigParam(..., default=True, ...)`). It outputs span names, timestamps, log levels, and span attributes to stdout using `rich`. Attributes from the `SAFE_KEYS` set — including `http.url`, `url.full`, `url.query`, `db.statement`, `exception.stacktrace`, `exception.message` — are printed verbatim.

  In shared environments (CI/CD pipelines with log aggregation, shared SSH sessions, container orchestration platforms where stdout is shipped to a central log store), this output may expose:

  - Full HTTP URLs including query strings (which may contain API keys or session tokens passed as query parameters)
  - SQL statements (with the same concerns as DS-02)

  - Exception messages and stack traces (which may include credential values in exception messages)

  While this is standard observability behaviour, the "on by default" setting means developers in shared environments may not realise they are printing sensitive span content to a potentially-shared output stream.

- **Affected Users**: Developers in shared CI environments or container platforms where stdout is aggregated.
- **Regulatory Reference**: General information security best practices.
- **Recommendation**:
  i. In the `ConsoleOptions` documentation, add a note recommending that `console=False` or `LOGFIRE_CONSOLE=false` be set in CI/CD environments where stdout is aggregated.
  ii. Consider applying the scrubbing pipeline to console output (currently the scrubber runs before the OTLP export, but the console exporter receives the *raw* span before scrubbing when using `SimpleSpanProcessor` — verify the processing order ensures scrubbing applies to console output as well). **If console output is not scrubbed, this is a Medium-severity finding.**
  iii. The `url.query` attribute should be considered for inclusion in the scrubbing pipeline's SAFE_KEYS review — query parameters often contain API keys or tokens.

Positive Safety Controls Present

The following controls are acknowledged as effective safety measures already implemented:

Control	Description	File
Built-in scrubbing	16 default patterns redact passwords, secrets, API keys, JWTs, SSNs, credit card numbers, and CSRF tokens from span attributes	`logfire/_internal/scrubbing.py`

Control	Description	File
`suppress_instrumentation()`	Context manager that prevents nested spans from being created	`logfire/_internal/utils.py`
`@no_auto_trace` decorator	Excludes specific functions from auto-tracing	`logfire/_internal/auto_trace/rewrite_ast.py`
`send_to_logfire=False`	Completely disables telemetry export	`logfire/_internal/config_params.py`
Pytest auto-disable	`PYTEST_VERSION` detection disables export by default in test environments	`logfire/_internal/config_params.py` (line 53)
Sampling controls	Head and tail sampling reduce data volume	`logfire/sampling/`
`min_level` filter	Suppresses low-level spans before they are created	`logfire/_internal/config.py`
Gzip compression	Reduces bandwidth for all OTLP exports	`logfire/_internal/config.py` (Compression.Gzip)
5 MB body limit	Limits individual request size, reducing exposure in transit	`logfire/_internal/exporters/otlp.py` (line 34)
Daemon thread for DiskRetryer	Prevents process hang on exit	`logfire/_internal/exporters/otlp.py` (line 163)
No photosensitive output	All console output is static Rich text, no animations or flashing	`logfire/_internal/exporters/console.py`
Token pattern validation	Write token format enforced by regex before use	`logfire/_internal/auth.py` (line 28)

Control	Description	File
Token not allowed in file config	`TOKEN` ConfigParam has `allow_file_config=False`, preventing accidental commit of tokens to pyproject.toml	`logfire/_internal/config_params.py` (line 61)
Scrubbing exempts span metadata	Internal logfire metadata keys (span type, message template, tags) are in SAFE_KEYS to prevent false-positive scrubbing	`logfire/_internal/scrubbing.py` (lines 113–160)
`NoopScrubber` class	Explicit noop implementation rather than None-checking throughout, making scrubbing bypass explicit	`logfire/_internal/scrubbing.py` (lines 172–183)

Remediation Priority

Priority	Finding	Effort	Impact
P1 (Immediate)	DS-01 — LLM content capture opt-in	Medium	High — prevents covert user data export
P1 (Immediate)	DS-03 — `scrubbing=False` warning	Low	High — low-cost safety net for a common footgun
P2 (Short-term)	DS-02 — `db.statement` scrubbing	Medium	High — prevents SQL PII leakage
P2 (Short-term)	DS-08 — Data collection disclosure	Low	Medium — improves developer transparency
P3 (Medium-term)	DS-05 — Pydantic plugin opt-in	Medium	Medium — reduces implicit footprint

Priority	Finding	Effort	Impact
P3 (Medium-term)	DS-04 — Metadata disclosure	Low	Medium — transparency improvement
P4 (Backlog)	DS-06 — Rate limiting	High	Medium — operational safety
P4 (Backlog)	DS-07 — DiskRetryer cleanup	Low	Medium — data retention hygiene
P5 (Documentation)	DS-09 — Console in CI	Low	Low — operational guidance

---

SDL Phase Mapping

Finding	SDL Phase	Rationale
DS-01	**Design**	Default capture behaviour is an architectural choice
DS-02	**Design**	Placement of `db.statement` in SAFE_KEYS is a design decision
DS-03	**Design**	Missing safety warning is a design oversight
DS-04	**Requirements**	Disclosure requirement needs to be added to requirements
DS-05	**Design**	Plugin activation model is an architectural decision
DS-06	**Requirements**	Rate-limiting controls need to be specified as requirements
DS-07	**Design**	Retention policy for retry files is a design concern
DS-08	**Requirements**	First-run disclosure is a requirements-level control
DS-09	**Design**	Console exporter default and scrubbing pipeline ordering

---

Report generated by Digital Safety Agent — SDL-aligned. Date: 2025-07-14.